Nama : Putra Sangaji

NPM : 06.2024.1.07750

Disini saya menggunakan RDBMS PostgreSQL V16.8 lalu Database saya disini adalah database yang akan menampung inventaris kantor beserta siapa pemiliknya, kapan dibelinya, serta history assetnya



Bagian 1 Dataset Dummy & Baseline Execution Plan

- Pembuatan DDL dan Data Dummy

  Karena jika saya copas scriptnya akan Panjang maka saya akan berikan link repo untuk melihat script **schema.sql** dan **db-seed.sql.** Repo ini juga berisi luaran dari tugas kali ini.

  Link Repository : https://github.com/christian1403/optek-basdat-inventarisku.git

- Baseline Measurement

  Query ini bertujuan untuk mencari jumlah server yang dibeli setiap bulan selama tahun 2023, yang saat ini berstatus Digunakan, berlokasi di Data Center Surabaya (DRC), dan memiliki minimal satu riwayat pada tabel asset_history

```
SELECT
   s.nama AS status,
   EXTRACT(MONTH FROM a.tanggal_pembelian) AS bulan_beli,
   COUNT(*) AS jml_server,
   SUM(a.harga_pembelian) AS total_nilai
FROM asset a
JOIN status s
   ON s.id = a.id_status
WHERE a.id_kategori = 5 AND s.id = 2
   AND a.id_lokasi = 4
   AND a.tanggal_pembelian >= DATE '2023-01-01'
   AND a.tanggal_pembelian < DATE '2024-01-01'
   AND (
```

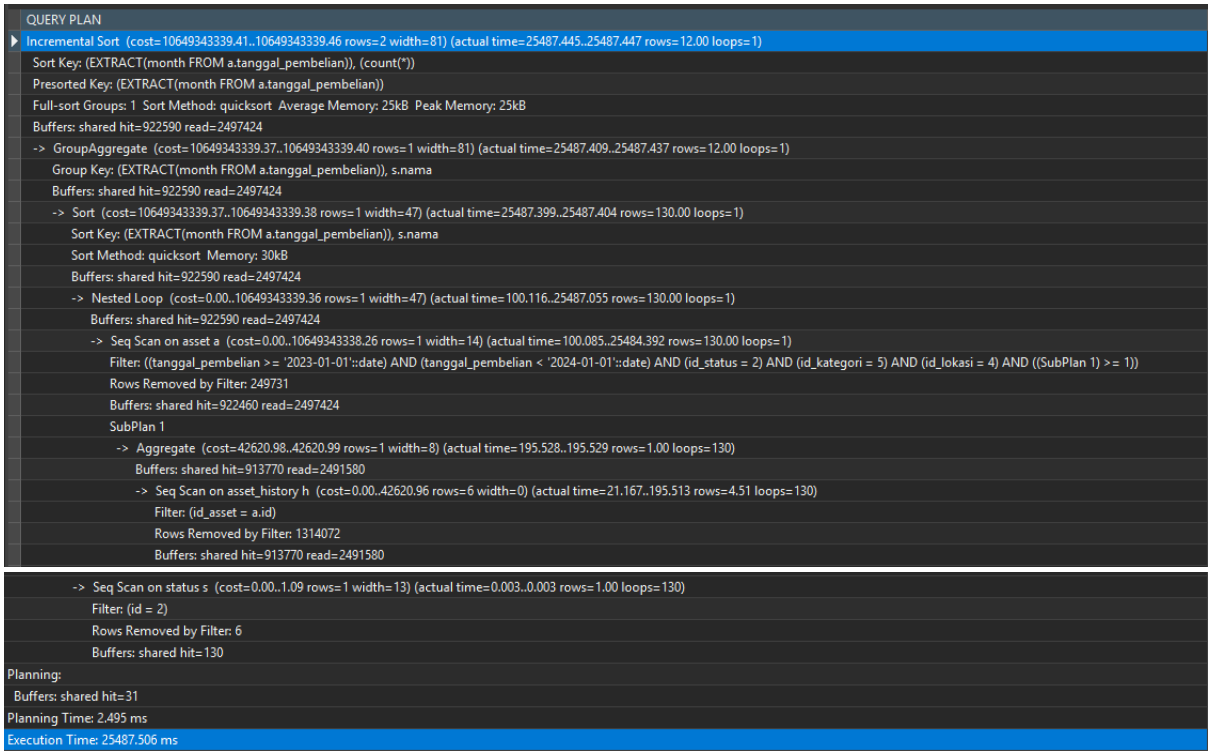
```
    SELECT COUNT(*)
    FROM asset_history h
    WHERE h.id_asset = a.id
  ) >= 1
GROUP BY
  s.nama,
  EXTRACT(MONTH FROM a.tanggal_pembelian)
ORDER BY
                bulan_beli ASC,
  jml_server;
```

- Analisis Execution Plan

  Disini saya menggunakan EXPLAIN (ANALYZE, BUFFERS) untuk mendapatkan Excecution plan

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  s.nama AS status,
  EXTRACT(MONTH FROM a.tanggal_pembelian) AS bulan_beli,
  COUNT(*) AS jml_server,
  SUM(a.harga_pembelian) AS total_nilai
FROM asset a
JOIN status s
  ON s.id = a.id_status
WHERE a.id_kategori = 5 AND s.id = 2
  AND a.id_lokasi = 4
  AND a.tanggal_pembelian >= DATE '2023-01-01'
  AND a.tanggal_pembelian < DATE '2024-01-01'
  AND (
    SELECT COUNT(*)
    FROM asset_history h
    WHERE h.id_asset = a.id
  ) >= 1
GROUP BY
  s.nama,
  EXTRACT(MONTH FROM a.tanggal_pembelian)
ORDER BY
                bulan_beli ASC,
  jml_server;
```

- Pencatatan Baseline



Terlihat dari hasil EXPLAIN (ANALYZE, BUFFERS), ditemukan ada **3 Sequential scan** yaitu pada table asset, asset_history, dan status.

Tabel	Rows	Rows Removed	Loops
Asset	130	249.731	1
Asset_history	4,51	1.314.072	130
Status	1	6	130

Terlihat juga total waktu eksekusi (Execution Time) adalah 25487.445 ms dengan total cost nya adalah 10.649.343.339,46 pada node tertinggi dan estimasi jumlah baris pada node adalah 2

Lalu titik bottleneck dari query tersebut ada pada table Asset_history Dimana postgresql melakukan scanning 1,3jt data dan diloop sebanyak 130x untuk mencari Riwayat berdasarkan id_asset

Bagian 2: Rancangan & Eksperimen Indeks

- **Strategi Pengindeksan**

  Untuk pengindeksan saya membuat 2 indeks yaitu

  - indeks B-Tree pada table asset_history

    ```
    CREATE INDEX idx_asset_history_asset
    ON asset_history (id_asset);
    ```

    indeks ini bertujuan mempercepat pencarian riwayat berdasarkan id aset

  - composite indeks pada table asset

    ```
    CREATE INDEX idx_asset_filter_laporan
    ON asset (id_kategori, id_status, id_lokasi, tanggal_pembelian);
    ```

    Indeks ini bertujuan untuk mempercepat pencarian data table asset berdasarkan kategori, status, Lokasi, dan tanggal_pembelian. Pada indeks ini saya memperhatikan prinsip leftmost prefix rule Dimana 3 kolom pertama menggunakan kondisi kesetaraan (=) sedangkann kolom terakhir menggunakan rentang tanggal

    2 indeks yang sudah kita buat bisa dicek ukuran atau sizenya dengan query :

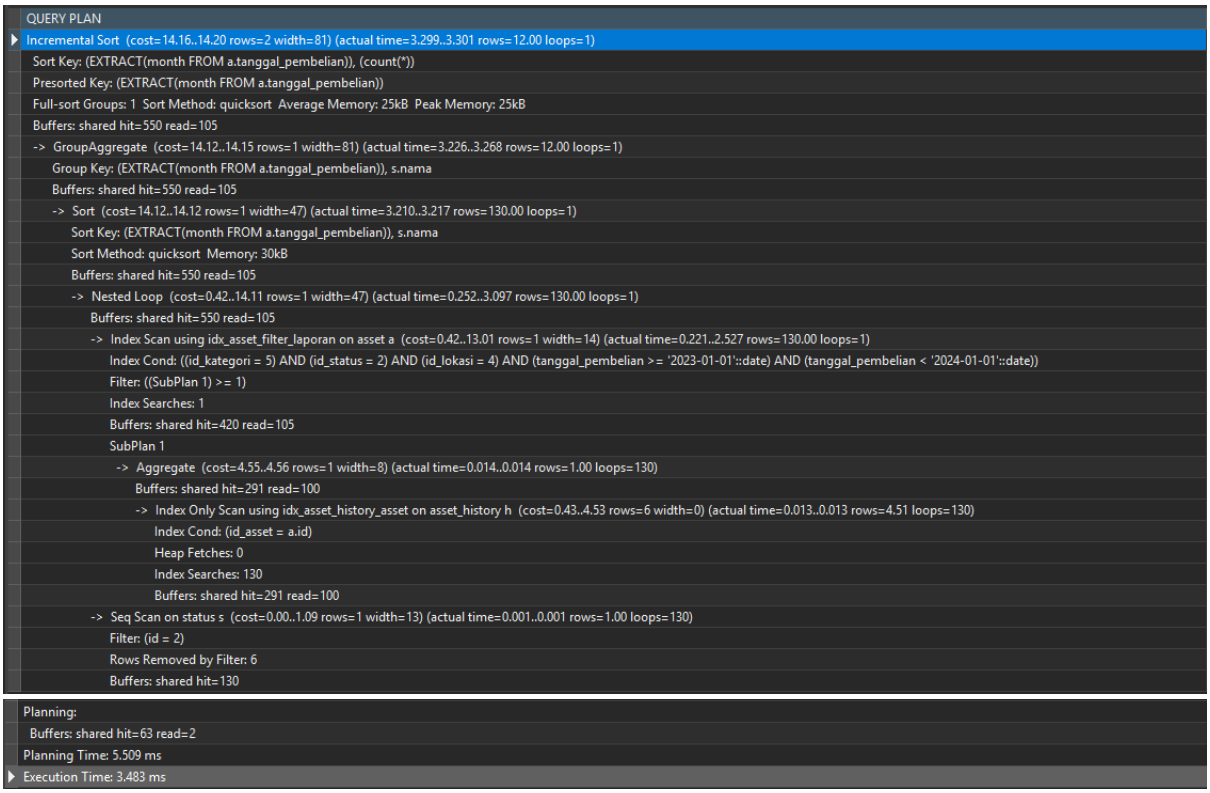
    ```
    SELECT
      indexrelname AS nama_indeks,
      pg_size_pretty(pg_relation_size(indexrelid)) AS ukuran_indeks,
      pg_relation_size(indexrelid) AS ukuran_byte
    FROM pg_stat_user_indexes
    WHERE indexrelname IN (
      'idx_asset_history_asset',
      'idx_asset_filter_laporan'
    )
    ORDER BY pg_relation_size(indexrelid) DESC;
    ```

    Maka akan menampilkan output size atau ukuran indeksnya

Nama Indeks	Ukuran
idx_asset_history_asset	15 mb
idx_asset_filter_laporan	6904 kb

- Evaluasi Perubahan

Ketika indeks selesai dibuat dan 2 tabel yang diindeks sudah di analyze berikut adalah perubahan yang terjadi kepada hasil query



Tabel Perbandingan Sesudah Dan Sebelum Ada Indeks

Aspek	**Sebelum**	**Sesudah**
Waktu Eksekusi	25.487 detik	1.298 ms
Estimasi Cost	10.649.343.339,46	14.20
Scan Pada asset	Seq Scan	Index Scan
Scan Pada asset_hitory	Seq Scan	Index Only Scan
Scan Pada status	Seq Scan	Seq Scan

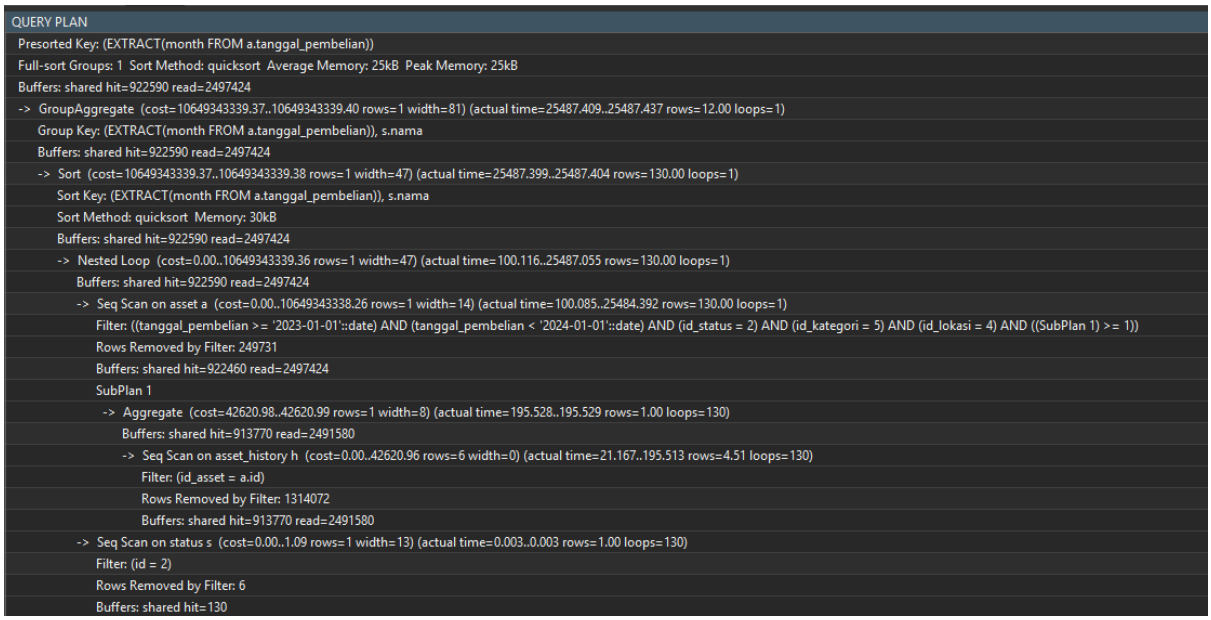
Bisa dilihat dari tabel perbandingan, pengindeksan berhasil mempercepat query. Pada hasil analyze sebelumnya seluruh pencarian menggunakan sequential search Dimana itu akan memakan banyak waktu, sedangkan Ketika indeks sudah dibuat pencarian pada table asset dan asset_history menggunakan

indeks yang sudah kita buat sebelumnya. Hasilnya terlihat jauh lebih cepat, dengan begitu makan pengindeksan dinyatakan berhasil

Bagian 3 : Pengujian Dampak JOIN, DML, & Pelaporan

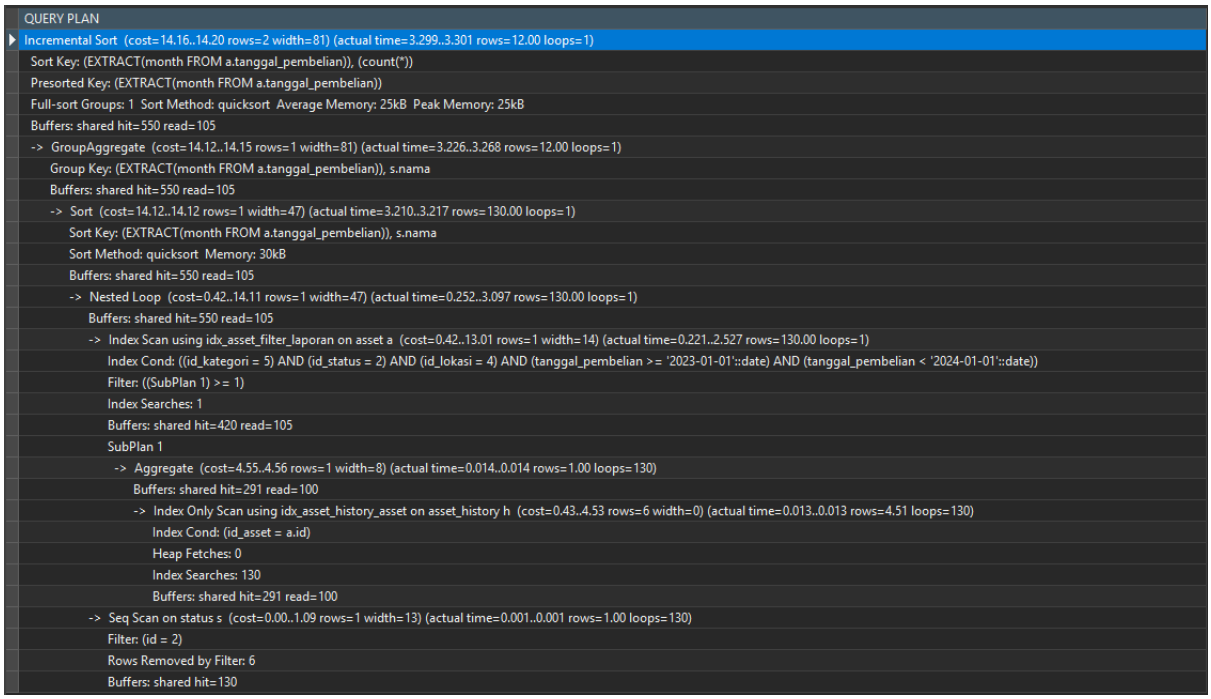
- Analisa Algoritma Join

Sebelum Indeks



```
QUERY PLAN
Presorted Key: (EXTRACT(month FROM a.tanggal_pembelian))
Full-sort Groups: 1  Sort Method: quicksort  Average Memory: 25kB  Peak Memory: 25kB
Buffers: shared hit=922590 read=2497424
-> GroupAggregate  (cost=10649343339.37..10649343339.40 rows=1 width=81) (actual time=25487.409..25487.437 rows=12.00 loops=1)
    Group Key: (EXTRACT(month FROM a.tanggal_pembelian)), s.nama
    Buffers: shared hit=922590 read=2497424
    -> Sort  (cost=10649343339.37..10649343339.38 rows=1 width=47) (actual time=25487.399..25487.404 rows=130.00 loops=1)
        Sort Key: (EXTRACT(month FROM a.tanggal_pembelian)), s.nama
        Sort Method: quicksort  Memory: 30kB
        Buffers: shared hit=922590 read=2497424
        -> Nested Loop  (cost=0.00..10649343339.36 rows=1 width=47) (actual time=100.116..25487.055 rows=130.00 loops=1)
            Buffers: shared hit=922590 read=2497424
            -> Seq Scan on asset a  (cost=0.00..10649343338.26 rows=1 width=14) (actual time=100.085..25484.392 rows=130.00 loops=1)
                Filter: ((tanggal_pembelian >= '2023-01-01'::date) AND (tanggal_pembelian < '2024-01-01'::date) AND (id_status = 2) AND (id_kategori = 5) AND (id_lokasi = 4) AND ((SubPlan 1) >= 1))
                Rows Removed by Filter: 249731
                Buffers: shared hit=922460 read=2497424
                SubPlan 1
                  -> Aggregate  (cost=42620.98..42620.99 rows=1 width=8) (actual time=195.528..195.529 rows=1.00 loops=130)
                      Buffers: shared hit=913770 read=2491580
                      -> Seq Scan on asset_history h  (cost=0.00..42620.96 rows=6 width=0) (actual time=21.167..195.513 rows=4.51 loops=130)
                          Filter: (id_asset = a.id)
                          Rows Removed by Filter: 1314072
                          Buffers: shared hit=913770 read=2491580
            -> Seq Scan on status s  (cost=0.00..1.09 rows=1 width=13) (actual time=0.003..0.003 rows=1.00 loops=130)
                Filter: (id = 2)
                Rows Removed by Filter: 6
                Buffers: shared hit=130
```

Sesudah Indeks



```
QUERY PLAN
Incremental Sort  (cost=14.16..14.20 rows=2 width=81) (actual time=3.299..3.301 rows=12.00 loops=1)
  Sort Key: (EXTRACT(month FROM a.tanggal_pembelian)), (count(*))
  Presorted Key: (EXTRACT(month FROM a.tanggal_pembelian))
  Full-sort Groups: 1  Sort Method: quicksort  Average Memory: 25kB  Peak Memory: 25kB
  Buffers: shared hit=550 read=105
  -> GroupAggregate  (cost=14.12..14.15 rows=1 width=81) (actual time=3.226..3.268 rows=12.00 loops=1)
      Group Key: (EXTRACT(month FROM a.tanggal_pembelian)), s.nama
      Buffers: shared hit=550 read=105
      -> Sort  (cost=14.12..14.12 rows=1 width=47) (actual time=3.210..3.217 rows=130.00 loops=1)
          Sort Key: (EXTRACT(month FROM a.tanggal_pembelian)), s.nama
          Sort Method: quicksort  Memory: 30kB
          Buffers: shared hit=550 read=105
          -> Nested Loop  (cost=0.42..14.11 rows=1 width=47) (actual time=0.252..3.097 rows=130.00 loops=1)
              Buffers: shared hit=550 read=105
              -> Index Scan using idx_asset_filter_laporan on asset a  (cost=0.42..13.01 rows=1 width=14) (actual time=0.221..2.527 rows=130.00 loops=1)
                  Index Cond: ((id_kategori = 5) AND (id_status = 2) AND (id_lokasi = 4) AND (tanggal_pembelian >= '2023-01-01'::date) AND (tanggal_pembelian < '2024-01-01'::date))
                  Filter: ((SubPlan 1) >= 1)
                  Index Searches: 1
                  Buffers: shared hit=420 read=105
                  SubPlan 1
                    -> Aggregate  (cost=4.55..4.56 rows=1 width=8) (actual time=0.014..0.014 rows=1.00 loops=130)
                        Buffers: shared hit=291 read=100
                        -> Index Only Scan using idx_asset_history_asset on asset_history h  (cost=0.43..4.53 rows=6 width=0) (actual time=0.013..0.013 rows=4.51 loops=130)
                            Index Cond: (id_asset = a.id)
                            Heap Fetches: 0
                            Index Searches: 130
                            Buffers: shared hit=291 read=100
              -> Seq Scan on status s  (cost=0.00..1.09 rows=1 width=13) (actual time=0.001..0.001 rows=1.00 loops=130)
                  Filter: (id = 2)
                  Rows Removed by Filter: 6
                  Buffers: shared hit=130
```

Algoritma Join tidak mengalami perubahan baik sebelum atau sesudah memakai indeks, disini indeks hanya mempercepat pencarian data yang diproses oleh join

- Analiss Overhead DML

INSERT

Disini saya akan coba insert ke table asset_history dengan menggunakan indeks dan tidak menggunakan indeks untuk mendapatkan perbandingan waktu saat insert

```
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, WAL)
INSERT INTO asset_history (id_asset, tanggal, id_jenis, id_status, keluhan,
tindakan, biaya)
SELECT
  (SELECT id FROM asset ORDER BY random() LIMIT 1),
  CURRENT_DATE - (random() * 700)::int, 1, 1,
  repeat('keluhan uji ', 5),
  repeat('tindakan uji ', 5),
  round((random() * 5000000)::numeric, 2)
FROM generate_series(1, 100000);
ROLLBACK;

--------------------------------------------------------------------------------

DROP INDEX IF EXISTS idx_asset_history_asset;
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, WAL)
INSERT INTO asset_history (id_asset, tanggal, id_jenis, id_status, keluhan,
tindakan, biaya)
SELECT
  (SELECT id FROM asset ORDER BY random() LIMIT 1),
  CURRENT_DATE - (random() * 700)::int, 1, 1,
  repeat('keluhan uji ', 5),
  repeat('tindakan uji ', 5),
  round((random() * 5000000)::numeric, 2)
FROM generate_series(1, 100000);
ROLLBACK;
```

Hasil Ketika tidak ada indeks

```
QUERY PLAN
Insert on asset_history  (cost=11114.30..17114.30 rows=0 width=0) (actual time=802.747..802.748 rows=0.00 loops=1)
  Buffers: shared hit=308281 dirtied=2906 written=2906
  WAL: records=203305 fpi=1 bytes=30817755 buffers full=2479
  ->  Subquery Scan on "*SELECT*"  (cost=11114.30..17114.30 rows=100000 width=1190) (actual time=46.339..236.060 rows=100000.00 loops=1)
        Buffers: shared hit=101369
        WAL: records=3030 bytes=299970 buffers full=31
        ->  Function Scan on generate_series  (cost=11114.30..14364.30 rows=100000 width=112) (actual time=46.326..155.516 rows=100000.00 loops=1)
              Buffers: shared hit=1369
              InitPlan 1
                ->  Limit  (cost=11114.29..11114.30 rows=1 width=12) (actual time=41.817..41.818 rows=1.00 loops=1)
                      Buffers: shared hit=1369
                      ->  Sort  (cost=11114.29..11738.95 rows=249861 width=12) (actual time=41.815..41.815 rows=1.00 loops=1)
                            Sort Key: (random())
                            Sort Method: top-N heapsort  Memory: 25kB
                            Buffers: shared hit=1369
                            ->  Index Only Scan using asset_pkey on asset  (cost=0.42..9864.99 rows=249861 width=12) (actual time=0.147..22.969 rows=249861.00 loops=
                                  Heap Fetches: 0
                                  Index Searches: 1
                                  Buffers: shared hit=1369
Planning:
  Buffers: shared hit=7
Planning Time: 1.073 ms
Trigger for constraint asset_history_id_asset_fkey: time=496.707 calls=100000
Trigger for constraint asset_history_id_jenis_fkey: time=481.272 calls=100000
Trigger for constraint asset_history_id_teknisi_fkey: time=61.375 calls=100000
Trigger for constraint asset_history_id_status_fkey: time=465.104 calls=100000
Trigger for constraint asset_history_id_sparepart_fkey: time=60.459 calls=100000
Execution Time: 2395.966 ms
```

Hasil Ketika pakai indeks

```
QUERY PLAN
Insert on asset_history  (cost=11114.30..17114.30 rows=0 width=0) (actual time=949.474..949.476 rows=0.00 loops=1)
  Buffers: shared hit=609817 read=5 dirtied=2995 written=2714
  WAL: records=304401 fpi=8 bytes=37352552 buffers full=2651
  ->  Subquery Scan on "*SELECT*"  (cost=11114.30..17114.30 rows=100000 width=1190) (actual time=41.210..225.255 rows=100000.00 loops=1)
        Buffers: shared hit=101369 dirtied=1
        WAL: records=3031 bytes=300069 buffers full=32
        ->  Function Scan on generate_series  (cost=11114.30..14364.30 rows=100000 width=112) (actual time=41.068..147.025 rows=100000.00 loops=1)
              Buffers: shared hit=1369
              InitPlan 1
                ->  Limit  (cost=11114.29..11114.30 rows=1 width=12) (actual time=37.373..37.374 rows=1.00 loops=1)
                      Buffers: shared hit=1369
                      ->  Sort  (cost=11114.29..11738.95 rows=249861 width=12) (actual time=37.371..37.372 rows=1.00 loops=1)
                            Sort Key: (random())
                            Sort Method: top-N heapsort  Memory: 25kB
                            Buffers: shared hit=1369
                            ->  Index Only Scan using asset_pkey on asset  (cost=0.42..9864.99 rows=249861 width=12) (actual time=0.106..20.576 rows=249861.00 loops=
                                  Heap Fetches: 0
                                  Index Searches: 1
                                  Buffers: shared hit=1369
Planning:
  Buffers: shared hit=28 read=1
Planning Time: 2.041 ms
Trigger for constraint asset_history_id_asset_fkey: time=489.784 calls=100000
Trigger for constraint asset_history_id_jenis_fkey: time=479.488 calls=100000
Trigger for constraint asset_history_id_teknisi_fkey: time=60.574 calls=100000
Trigger for constraint asset_history_id_status_fkey: time=464.321 calls=100000
Trigger for constraint asset_history_id_sparepart_fkey: time=60.687 calls=100000
Execution Time: 2532.734 ms
```

Terdapat perbedaan execution time antara proses INSERT tanpa indeks dan menggunakan indeks. Berdasarkan hasil pengujian, proses INSERT tanpa indeks membutuhkan waktu 2.395,966 ms, sedangkan proses INSERT menggunakan indeks membutuhkan waktu 2.532,734 ms

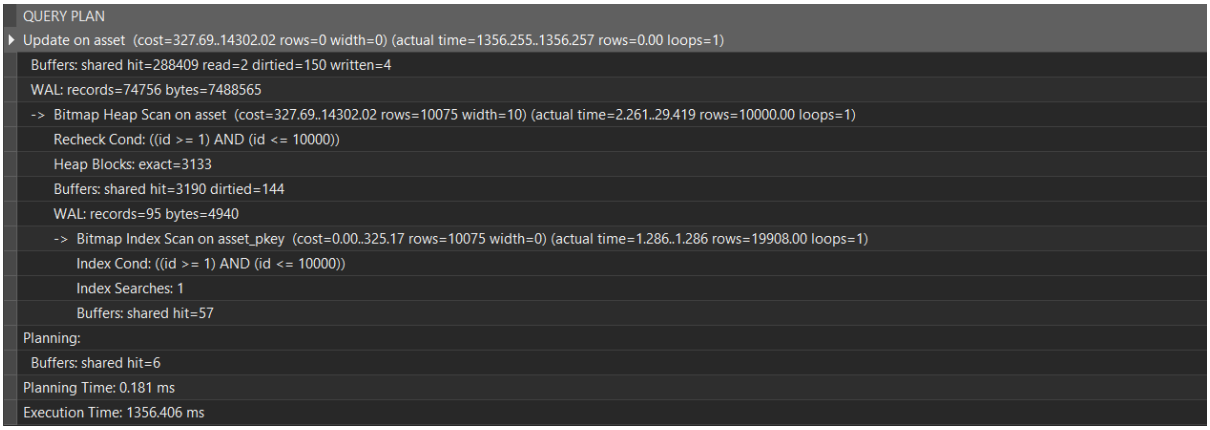
Tabel Perbandingan Execution time		
Tanpa Indeks	Menggunakan Indeks	Selisih
2395,966 ms	2532,734 ms	136,768 ms

Perbedaan ini dikarenakan postgresql mendapatkan pekerjaan tambahan Ketika memakai indeks yaitu memperbarui struktur indeks dengan memasukkan id_asset dari setiap baris baru

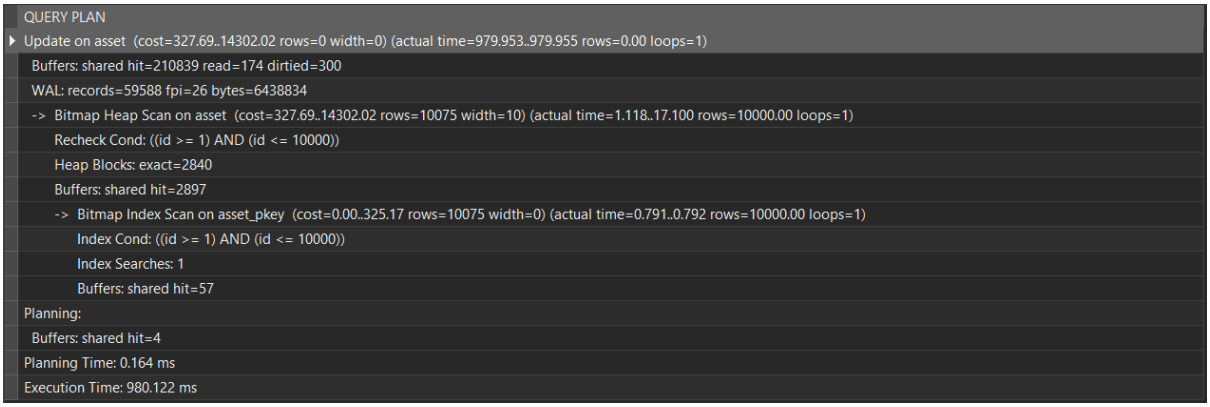
UPDATE

```
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, WAL)
UPDATE asset
SET id_lokasi = id_lokasi
WHERE id BETWEEN 1 AND 10000;
ROLLBACK;
----------------------------------------------------------------
DROP INDEX IF EXISTS idx_asset_filter_laporan;
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, WAL)
UPDATE asset
SET id_lokasi = id_lokasi
WHERE id BETWEEN 1 AND 10000;
ROLLBACK;
```

Hasil Ketika memakai indeks



Hasil Ketika tidak memamakai indeks



Terdapat perbedaan execution time antara proses UPDATE tanpa indeks dan menggunakan indeks. Berdasarkan hasil pengujian, proses UPDATE tanpa indeks membutuhkan waktu 1356,406 ms, sedangkan proses UPDATE menggunakan indeks membutuhkan waktu 980, 122 ms

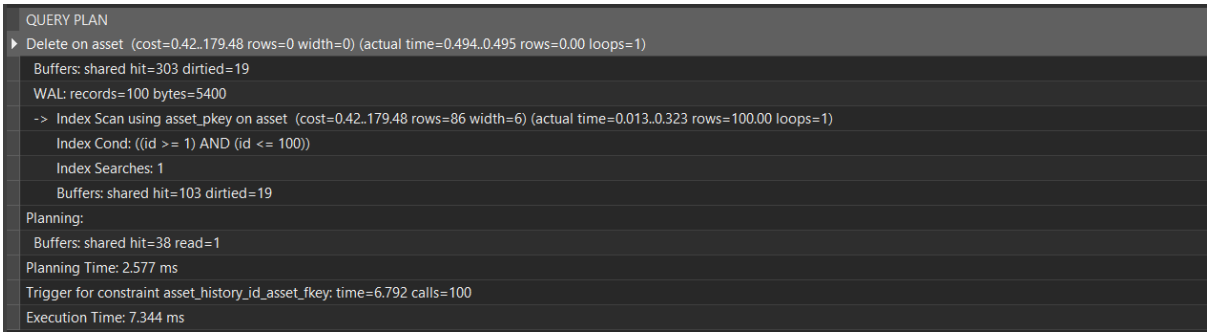
Tabel Perbandingan Execution time		
Tanpa Indeks	Menggunakan Indeks	Selisih
1356,406 ms	980,112 ms	372,284 ms

Namun bisa dilihat juga kedua execution plan masih menggunakan indeks primary key asset_pkey, sehingga hasil ini belum dapat digunakan untuk menyimpulkan secara pasti bahwa indeks tambahan menyebabkan operasi UPDATE menjadi lebih lambat

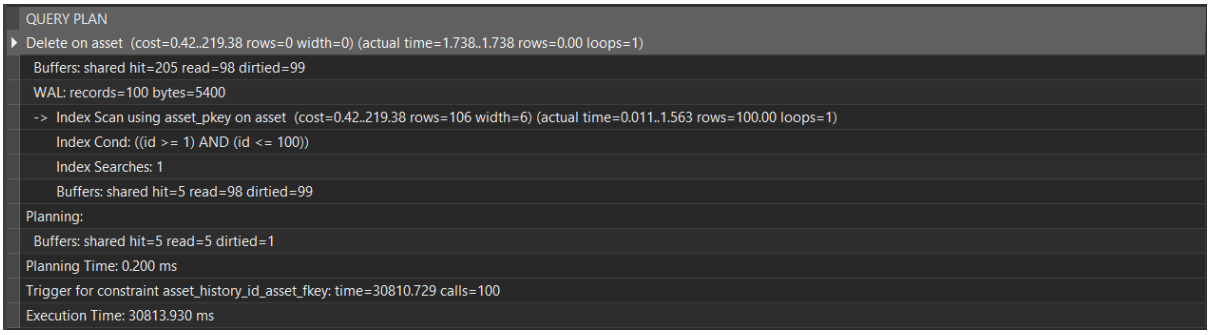
DELETE

```
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, WAL)
DELETE FROM asset
WHERE id BETWEEN 1 AND 100;
ROLLBACK;
-------------------------------------------------
DROP INDEX IF EXISTS idx_asset_filter_laporan;
DROP INDEX IF EXISTS idx_asset_history_asset;
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, WAL)
DELETE FROM asset
WHERE id BETWEEN 1 AND 100;
ROLLBACK;
```

Hasil Ketika memakai indeks



Hasil Ketika tidak memakai indeks



Terdapat perbedaan execution time antara proses DELETE tanpa indeks dan menggunakan indeks. Berdasarkan hasil pengujian, proses DELETE tanpa indeks membutuhkan waktu 30813,930 ms, sedangkan proses DELETE menggunakan indeks membutuhkan waktu 7.344 ms

Tabel Perbandingan Execution time		
Tanpa Indeks	Menggunakan Indeks	Selisih
30813,930 ms	7.344 ms	30806,586 ms

Perbedaan terbesar terlihat pada waktu pemeriksaan foreign key asset_history_id_asset_fkey. Hal ini menunjukkan bahwa indeks pada kolom asset_history.id_asset membantu PostgreSQL memeriksa referensi aset secara lebih efisien ketika proses DELETE dilakukan